Chapter 43 - Coprocessor Positive Cookbook

Chapter 32 explains the coprocessor registers and the failure path. This chapter shows the positive path: put a tiny worker image in guest RAM, start it, enqueue a request, wait for completion, inspect the shared mailbox, then stop the worker.

The worker in this chapter is a 6502 service. It does not use a hidden file. BASIC enters the 6502 bytes into memory and starts them with `COPROC_CMD_START_MEM`.

43.1 The Mailbox Shape

Each worker has a ring inside the shared mailbox at `$790000`. The 6502 worker is CPU type `3`, with `cpuTypeIndex` `1` and instance `0`, so its ring index is `1 * 2 + 0 = 2` at the uniform `$400` stride:

```
$790000 + 2 * $400 = $790800
```

Inside the 6502 worker address space, that same ring appears at `$2800`, because the worker maps the mailbox at CPU address `$2000`.

Field	Bus address	6502 address
Ring head	`$790800`	`$2800`
Ring tail	`$790801`	`$2801`
Ring capacity	`$790802`	`$2802`
Request slot `0`	`$790808`	`$2808`
Response slot `0`	`$790A08`	`$2A08`
Response result code	`$790A10`	`$2A10`

The example assumes a fresh worker and sends one request, so slot `0` is enough.

43.2 The 6502 Service Bytes

This service waits until `head` and `tail` differ. It reads the request operation byte from request slot `0`, adds `1`, writes that value into the response descriptor result-code field, sets response status to `COPROC_TICKET_OK`, advances the tail to `1`, and waits for more work.

```
AD 03 28  8D 04 28
AD 00 28  CD 01 28  F0 F8  AD 10 28  18 69 01  8D 10 2A
A9 00     8D 11 2A  8D 12 2A  8D 13 2A
8D 14 2A  8D 15 2A  8D 16 2A  8D 17 2A
A9 02     8D 0C 2A
A9 00     8D 0D 2A  8D 0E 2A  8D 0F 2A
A9 01     8D 01 28  4C 06 00
```

The same bytes as 6502 instructions:

```
$0000  LDA $2803       ; layout version
$0003  STA $2804       ; acknowledge version
$0006  LDA $2800       ; head
$0009  CMP $2801       ; tail
$000C  BEQ $0006
$000E  LDA $2810       ; request op, low byte
$0011  CLC
$0012  ADC #$01
$0014  STA $2A10       ; response result code low byte
$0017  LDA #$00
$0019  STA $2A11
$001C  STA $2A12
$001F  STA $2A13
$0022  STA $2A14       ; response length = 0
$0025  STA $2A15
$0028  STA $2A16
$002B  STA $2A17
$002E  LDA #$02
$0030  STA $2A0C       ; response status = OK
$0033  LDA #$00
$0035  STA $2A0D
$0038  STA $2A0E
$003B  STA $2A0F
$003E  LDA #$01
$0040  STA $2801       ; tail = 1
$0043  JMP $0006
```

43.3 Type The Positive Example

This BASIC listing enters the service bytes, starts the worker from guest RAM, enqueues operation 7, waits for completion, and inspects the response descriptor.

```
10 REM 6502 COPROCESSOR POSITIVE PATH
20 S=MEMALLOC(70,16)
30 FOR I=0 TO 69:READ B:POKE8 S+I,B:NEXT
40 POKE32 &H000F2344,3
50 POKE32 &H000F235C,S
60 POKE32 &H000F2360,70
70 POKE32 &H000F2340,6
80 PRINT "START ";PEEK32(&H000F2348),PEEK32(&H000F234C)
90 REQ=MEMALLOC(4,4):RESP=MEMALLOC(4,4)
100 POKE32 REQ,&H12345678
110 T=COCALL(3,7,REQ,4,RESP,4)
120 PRINT "TICKET ";T
130 COWAIT T,1000
140 PRINT "STATUS ";COSTATUS(T)
150 PRINT "RESULT ";PEEK32(&H00790A10)
160 COSTOP 3
170 DATA &HAD,&H03,&H28,&H8D,&H04,&H28,&HAD,&H00
180 DATA &H28,&HCD,&H01,&H28,&HF0,&HF8,&HAD,&H10
190 DATA &H28,&H18,&H69,&H01,&H8D,&H10,&H2A,&HA9
200 DATA &H00,&H8D,&H11,&H2A,&H8D,&H12,&H2A,&H8D
210 DATA &H13,&H2A,&H8D,&H14,&H2A,&H8D,&H15,&H2A
220 DATA &H8D,&H16,&H2A,&H8D,&H17,&H2A,&HA9,&H02
230 DATA &H8D,&H0C,&H2A,&HA9,&H00,&H8D,&H0D,&H2A
240 DATA &H8D,&H0E,&H2A,&H8D,&H0F,&H2A,&HA9,&H01
250 DATA &H8D,&H01,&H28,&H4C,&H06,&H00
```

Expected result: `START` is followed by status `0` and command error `0`, `TICKET` is non-zero, `STATUS` is `2`, and `RESULT` is `8`. The exact ticket number is not fixed. It only has to be non-zero. The result is `8` because the request operation was `7` and the worker adds `1`.

43.4 What The Lines Do

Lines `20` to `30` allocate public low memory and enter the 6502 image. Lines `40` to `70` start a CPU type `3` worker from that memory by writing the raw coprocessor MMIO command `6`.

The service's first two instructions copy the ring layout version from `$2803` to `$2804`. This acknowledgement is required before the start command succeeds and before the manager routes requests to the worker.

The stores from `$0014` through `$003B` finish the response result, length, and status fields. The store at `$0040` advances `tail` only after those fields are complete. That final byte is the publication step: once the caller sees the new tail, it may consume the response. Moving the tail store earlier could expose a response whose later fields still belong to the preceding operation. The ordering of these stores is what makes the complete descriptor ready.

Line `110` enqueues a normal BASIC `COCALL`. BASIC checks that `REQ` and `RESP` are public buffers created by `MEMALLOC`.

Line `130` waits for the worker. Line `140` prints the ticket status: `2` means `COPROC_TICKET_OK`. Line `150` reads the response descriptor inside the mailbox. This example uses the descriptor result code rather than a response byte buffer, because that keeps the 6502 service short and makes the mailbox visible to the reader.

43.5 Side Effects and Limits

- A running worker remains online until `COSTOP` or until the worker exits.
- `COCALL` returns ticket `0` if enqueue fails. Read `COPROC_CMD_ERROR` at `$F234C` for the command error.

- `COWAIT` uses the timeout in milliseconds. If it times out, `COSTATUS(ticket)` reports `4`.
- This first service handles the first request slot only. A full service must use the current tail value to choose a request and response slot.
- The 6502 worker sees its own `64` KB service memory plus the mailbox window. It does not automatically see every BASIC buffer by 16-bit address.

Chapter 44 shows how to debug this kind of worker with IE Mon and IE Script.